

4.5 — Design Efficient Batch Processing Strategies

Concept

The Anthropic Message Batches API allows you to submit large volumes of requests in a single call and receive results asynchronously. Batch processing is priced at 50% of the standard per-token cost, making it the correct choice whenever your workload can tolerate a non-real-time response window — up to 24 hours per batch.

When to Use Batches vs Real-Time

Use the Batches API for **non-interactive workloads**: document summarization pipelines, bulk sentiment analysis, dataset annotation, offline evaluation runs, and any job where the result is not needed synchronously by a waiting user or process.

Use the standard Messages API for **interactive workloads**: chat interfaces, tool-calling agents that need to act on each response, streaming UI updates, and any flow where latency matters.

The decision rule is simple: if a human or dependent process is waiting for the response in real time, use the Messages API. If the results feed a downstream pipeline that runs hours later, use batches.

`custom_id` — Request Tracking

Every request in a batch must include a `custom_id` — a caller-assigned string of up to 64 characters. The API preserves this identifier in the response, giving you a stable mapping between what you submitted and what you received. Without `custom_id`, correlating results across a large batch would require positional assumptions that break if individual requests fail.

Use a value that encodes enough context to act on the result without re-querying your database: `doc-{uuid}`, `user-{id}-review-{timestamp}`, or similar.

Latency Tolerance

Batches complete within 24 hours, but in practice most finish within one to four hours. Design your system to treat batch completion as an event, not a blocking call. Poll the batch status endpoint at a reasonable interval (every 60 seconds is sufficient for most workloads), or implement a webhook-triggered processor if your infrastructure supports it.

Do not submit a batch and immediately poll in a tight loop — you will waste API quota and gain nothing.

Failure Handling for Individual Items

Batch execution is per-item: a single failed request does not cancel the entire batch. Each result object carries its own `type` field — either `"succeeded"` or `"errored"`. An errored result includes an `error` object with a `type` and `message`. Your processing loop must check the result type for every item and handle errors explicitly, because there is no aggregate failure signal.

Common error types include `invalid_request` (malformed prompt or parameter), `overloaded` (service capacity), and `expired` (batch exceeded the 24h window before that item was processed).

Batch Lifecycle

1. **Create** — POST a batch with up to 10,000 requests. Receive a `batch_id`.
2. **Poll** — GET the batch status until `processing_status` is `"ended"`.
3. **Retrieve results** — GET the results stream; parse each newline-delimited JSON object.
4. **Handle failures** — Separate succeeded results from errored ones and decide whether to retry errored items in a new batch.

Code Example

Creating a Batch

```
import Anthropic from "@anthropic-ai/sdk";

const client = new Anthropic();

// Build requests — each needs a unique custom_id
const documents = [
  { id: "doc-001", text: "Climate change is accelerating sea level rise..." },
  { id: "doc-002", text: "New study links sleep deprivation to cognitive..." },
  { id: "doc-003", text: "Central banks signal further rate adjustments..." },
];

const requests = documents.map((doc) => ({
  custom_id: doc.id,
  params: {
    model: "claude-opus-4-5",
    max_tokens: 128,
    messages: [
      {
        role: "user" as const,
        content: `Classify the topic of this text in one word: ${doc.text}`,
      },
    ],
  },
}));

const batch = await client.messages.batches.create({ requests });
console.log("Batch created:", batch.id, "| Status:", batch.processing_status);
```

Polling for Completion

```

async function waitForBatch(batchId: string): Promise<Anthropic.MessageBatch> {
  while (true) {
    const batch = await client.messages.batches.retrieve(batchId);

    if (batch.processing_status === "ended") {
      return batch;
    }

    console.log(
      `Batch ${batchId} still processing – waiting 60s (${batch.request_counts.processing} remaining)`
    );
    await new Promise((resolve) => setTimeout(resolve, 60_000));
  }
}

const completedBatch = await waitForBatch(batch.id);
console.log("Request counts:", completedBatch.request_counts);

```

Processing Results with Failure Handling

```

const succeeded: Array<{ customId: string; classification: string }> = [];
const failed: Array<{ customId: string; error: string }> = [];

for await (const result of await client.messages.batches.results(batch.id)) {
  if (result.result.type === "succeeded") {
    const text = result.result.message.content[0];
    if (text.type === "text") {
      succeeded.push({ customId: result.custom_id, classification: text.text });
    }
  } else if (result.result.type === "errored") {
    failed.push({
      customId: result.custom_id,
      error: result.result.error.type,
    });
  }
}

console.log(`Succeeded: ${succeeded.length} | Failed: ${failed.length}`);

// Retry failed items in a new batch if warranted
if (failed.length > 0) {
  console.log("Failed custom_ids:", failed.map((f) => f.customId));
}

```

Common Mistakes

Using batches for interactive workloads. If a UI component is waiting for a response, batches will break your user experience. Batches are for offline pipelines only.

Ignoring the `type` field on each result. Iterating over results and assuming all are successes causes silent data loss. Always branch on `result.result.type` before accessing the message payload.

Tight polling loops. Polling every second wastes compute and quota. A 60-second interval is adequate for most workloads; for large annotation jobs, poll every 5 minutes.

Reusing `custom_id` values across batches. The API scopes `custom_id` within a single batch. To trace results back to your source records across multiple batches, encode the batch identifier or a stable record ID in your `custom_id`.

Assuming batches complete instantly. Code that submits a batch and expects results within seconds will produce false timeouts. Build your system with a job-queue model: submit, persist the `batch_id`, and process results in a separate scheduled task.

Not persisting the `batch_id`. If your process crashes after submission, you lose the ability to retrieve results unless you stored the `batch_id`. Always write it to a durable store before proceeding.

References

- Anthropic docs: [Message Batches API](#)
- Anthropic SDK: `client.messages.batches.create`, `.retrieve`, `.results`
- Pricing: Batch requests are billed at 50% of standard Messages API rates
- Exam objectives: Section 4 — Cost Optimization and Throughput Strategies